COMP2000 Logbook

Name: James Turnbull
Student ID: 49162713
Project: Predator and Prey Ecosystem
Repository: github.com/JamesTurnbull07/COMP2000Assignment

Week 3

10/8 Objects and classes
This week we decided on creating a predator/ prey ecosystem. We planned it out on paper (as seen on Github) and drew out the class design. I also brought up the idea of a safe zone. This safe zone would make it so predators can't eat prey when they are within it. This was to add another element to the project.
The basic idea at this point is that we will have entities, a safe zone, food and some sort of collision detection. The rules are that X will be controlled. If X touches Y it will disappear. If Y touches food it will double.

Week 4

17/8 Inheritance and overloading
This week we looked at implementing inheritance onto our entities. We discussed the use of different animals to implement and decided to use many such as rabbits, mice, hawks and foxes. All these animals will share most behaviors so we decided to have a larger animal class rather than rewriting the code for each one. Each animal can then overwrite the elements they need to to make them unique.

Week 5

24/8 Generics and type erasure
This week we worked on the Container/Main exercise and tried to make the program trigger a runtime error while avoiding explicit typecasting. My group worked on this together in class. We tried adding a null item and putting different data types in the same list. Both of these attempts did not work. I ended up having to leave the class around 20 minutes before it finished and did not catch the solution in class. Working on this further at home I found the best way to make this happen was to store the Container using its raw type.

Week 6

31/8 Exceptions and call stacks.
I was not present for the week 6 class as I was sick and ended up doing it at home on my own. I wrote the four chained methods and first ran them with nothing thrown, which printed the full pattern with no gaps. I then made chuck_a_fit throw an exception with nothing catching it, and every exit print after that point disappeared, since the exception just kept going up through the call stack. Then I added a catch inside pipe, and pipe's own exit print came back, but carets was still missing, because the exception had already stopped caret from finishing before pipe caught it.

Week 7

7/9 patterns
I was not present for this class as I worked an overnight shift the night before and forgot to set my alarm and ended up waking up late. This meant I was not able to present with my group. After the class I ended up speaking to my group. They told me that they presented our project. They also spoke of other teams most notably a covid simulation. They also mentioned a hangman game.

Week 8

28/9
I was not able to make it to class this week so I went through the material at home. I decided that the best pattern to use for our project is the strategy pattern. This gives every animal a movement strategy, meaning that new kinds of movements don't need new subclasses. I also looked at using the state pattern.

Week 9

5/10
This week my class fell on a public holiday. However I still looked at the lambda material. Since our movement strategy only has 1 method I decided that we will write each movement as a lambda instead of a class. This would make each movement 1 line of code instead of a whole method.

Week 10

12/10 Streams
This week we looked at replacing loops with streams in our project. I decided to replace the findNearest loop with a stream to filter out the dead animals. It also keeps track of the closest animal to it. I tested that this didn't break anything by running the program before and after and saw no drastic changes.

Week 11

19/10 Collectors
This week we looked at how collectors could be used in our project. I decided to use one to count the amount of animals that die in our simulation and what they died from. I also will add a counter for old age deaths as well as a new feature that shows how well fed each species is as well as its average health and which one is the hungriest.

Week 12

26/10 parallelism
This week we looked at how parallelism can be used in our projects. We made test runs and ran the simulation 6 times. All tests were run at the same time. Each test did not affect the others in any way. However, the results were printed in a random order so I needed to make each run hand back its result and print them all in order at the end of the testing.

Week 13

This week was the final showcase and gave people a chance to present their simulations. I wasn't able to attend and I did not see any other group's simulations.

Final Reflection

The main trade-off I faced was strategy versus subclasses. Strategy made movement more flexible, but not everything fit. The hawk’s resting depends on its own counter, so it stayed in the Hawk class. I resolved this by only using the strategy for behaviour that every animal has.
What surprised me most was testing the safe zone. I expected it to help the animals survive longer, but in 2 of 3 runs the simulation ended about twice as fast with it on. I think this is because prey hiding in the safe zone starve the predators.
I’m proud of the safe zone since it was my idea in week 3, and of adding SuperFood, death by old age and the fed column.

If I started over I would go to more of the classes and keep up with the project each week instead of doing a lot at the end. I would also use branches from the start instead of working straight on main.
The feedback I acted on from week 7 was that my log book was too descriptive, so this time I tried to explain why I made each decision and what the trade-offs were, like choosing Strategy over State.